

A. Related Work

A.1. Spatiotemporal Learning

Spatiotemporal learning has emerged as a fundamental area of research, addressing the need to model and interpret data that evolve across both space and time. This field has far-reaching applications in diverse scientific and engineering disciplines, enabling advancements in climate modeling (Reichstein et al., 2019), traffic forecasting (Li et al., 2018), medical imaging (Litjens et al., 2017), and video analysis (Wang et al., 2020). Beyond these areas, spatiotemporal learning plays a crucial role in biochemistry, where modeling molecular interactions and protein dynamics requires capturing complex temporal dependencies in high-dimensional spatial structures (Jumper et al., 2021). In nuclear particle physics, tracking subatomic particles in high-energy collisions demands precise spatiotemporal reconstruction to infer particle trajectories and decay chains (Aad et al., 2019). Neuroscience increasingly relies on spatiotemporal models to analyze large-scale neural recordings, such as EEG and fMRI, for understanding brain activity over time and across different brain regions (Van Essen et al., 2010). Similarly, in epidemiology, disease spread modeling depends on accurately capturing transmission dynamics across spatially distributed populations over time (Balcan et al., 2009).

Other critical applications include remote sensing and Earth observation, where satellite imagery and geospatial data must be processed to track environmental changes, deforestation, and urbanization trends (Woodcock et al., 2008). In fluid dynamics, understanding turbulent flow patterns and their evolution over time is key to designing efficient aerodynamic structures and predicting oceanic and atmospheric circulation (Meneveau & Marusic, 2011). These applications highlight the growing importance of spatiotemporal learning in scientific discovery and engineering innovation. However, effectively capturing the complex dependencies inherent in spatiotemporal data presents significant challenges, including spatial heterogeneity (irregularly structured and non-uniformly distributed data), temporal non-stationarity (changes in statistical properties over time), and high dimensionality (large-scale data with intricate interdependencies). Addressing these challenges requires scalable, generalizable, and computationally efficient modeling approaches that can learn from noisy, sparse, and dynamically evolving spatiotemporal datasets.

A.2. Traditional Approaches to Spatiotemporal Learning

Historically, Recurrent Neural Networks (RNNs) and Long Short-Term Memory networks (LSTMs) have been employed to model temporal sequences (Hochreiter & Schmidhuber, 1997). For instance, LSTMs have been widely used in traffic forecasting to predict future traffic conditions based on historical data (Yu et al., 2017). Despite their effectiveness, these models often struggle with capturing long-range dependencies and may not fully exploit spatial correlations. To address these limitations, Transformer-based architectures have been introduced, leveraging self-attention mechanisms to model long-range dependencies in both spatial and temporal dimensions. Vision Transformer (ViT) has demonstrated success in image analysis by treating images as sequences of patches, enabling the modeling of global relationships (Dosovitskiy et al., 2021). Extending this idea, TimeSformer has been proposed for video understanding, jointly modeling spatial and temporal dependencies to achieve state-of-the-art results in action recognition tasks (Bertasius et al., 2021).

A.3. Implicit Neural Representations (INRs) for Learning Continuous Representations

Implicit Neural Representations (INRs) have emerged as a flexible and powerful approach for modeling continuous signals. Unlike traditional grid-based representations, INRs parameterize data as continuous functions, mapping spatial coordinates to function values using a neural network (Sitzmann et al., 2020b). This formulation allows for compact data encoding, resolution-agnostic modeling, and seamless interpolation. INRs have been widely applied in 3D shape representation, scene reconstruction, and signal processing, where high-dimensional structured data needs to be represented efficiently. A notable example is Neural Radiance Fields (NeRF), which employs INRs for view synthesis, enabling the rendering of high-fidelity 3D scenes from sparse observations (Mildenhall et al., 2020).

Despite these advantages, INRs face challenges in generalizability, as a trained INR typically encodes only a single instance of data and does not naturally adapt to new instances. This limitation necessitates retraining the model for each new data sample, making INRs computationally expensive for large-scale applications. Generalizable INRs (GINRs) aim to overcome this by introducing mechanisms that allow a single model to adapt across multiple instances, rather than learning a fixed function for each data sample. Locality-Aware Generalizable Implicit Neural Representations introduce local feature conditioning that allows GINRs to adapt dynamically to different regions of a dataset, improving efficiency and generalization (Lee et al., 2023). Furthermore, MetaSDF and MetaSIREN employ gradient-based meta-learning to

enable few-shot adaptation, allowing GINRs to learn priors across multiple data instances and generalize to unseen samples more effectively (Sitzmann et al., 2020a; Chen et al., 2021). Functa treats each data instance as a function and leveraging function-space representations for improved generalization (Dupont et al., 2022). Spatial Functa extends this approach to large-scale datasets like ImageNet, introducing spatially aware latent spaces that enhance expressivity and enable GINRs to perform large-scale classification and generation tasks (Bauer et al., 2023).

A.4. Conditioned Neural Fields as GINRs

Recent work by Wang et al. (2024)(Wang et al., 2024) highlights the close relationship between CNFs and INRs, emphasizing that both frameworks model continuous fields but differ in their conditioning mechanisms. While INRs typically encode static signals without external conditioning, CNFs introduce input-dependent variations, making them well-suited for physics-informed learning in PDE solving. This connection unifies perspectives on operator learning and neural field-based modeling, bridging the gap between classical numerical methods and neural representations.

Conditioned Neural Fields (CNFs) provide a powerful framework for solving Partial Differential Equations (PDEs) by learning continuous function representations conditioned on input parameters, initial conditions, or constraints (Li et al., 2022). Unlike traditional numerical solvers that rely on discretization, CNFs approximate time-evolving fields in a resolution-independent manner, making them particularly effective for high-dimensional and sparse-data problems. This approach aligns closely with GINRs, which parameterize data as continuous functions rather than grid-based representations (Sitzmann et al., 2020b).

A.5. Baseline Selection

Spatiotemporal learning requires modeling dynamic systems such as fluid flows, climate forecasting, and wave propagation, all of which are governed by PDEs. PDE-based neural solvers provide strong priors that enhance generalization, consistency, and interpretability. To evaluate our approach (SCENT) against established methods, we compare against FNO, which learns spectral representations for PDE solutions, OFormer, a Transformer-based operator learner, and CORAL and AROMA, which leverage neural fields for dynamic system modeling. These baselines offer diverse perspectives on how different architectures generalize across spatiotemporal interpolation, reconstruction, and forecasting tasks.

Table 4. Comparing model capacities for learning spatiotemporal continuity from discrete data.

MODEL	MESH AGNOSTIC LEARNING	SPACE&TIME CONTINUOUS LEARNING	SINGLE-STEP TRAINING
FNO (LI ET AL., 2020)	✗	✗	✓
OFORMER (LI ET AL., 2023)	✓	✗	✓
CORAL (SERRANO ET AL., 2023)	✓	✓	✗
AROMA (SERRANO ET AL., 2024)	✓	✗	✗
SCENT (OURS)	✓	✓	✓

B. SCENT: Pseudo-Algorithm

We omit spatiotemporal mesh variables $\left(\{\mathbf{x}_j\}_{j=1}^{N_i}, t_i, t_o\right)$ in the algorithm for simplification.

Algorithm 1 Training and Inference for Model F

Training Procedure

Require: Model F , training dataset $\mathcal{D}$ with n_{tr} trajectories, each of T timesteps

Require: Time horizon t_h , total training steps N_{train} , loss function g

Initialize model parameters θ

for iteration $i \leftarrow 1$ to N_{train} **do**

 Select i th trajectory from $\mathcal{D}$

 Sample (U^{t_i}, U^{t_o}) with time horizon t_h , where $t_o - t_i = \Delta t \in \{0, 1, 2, \dots, t_h\}$

Forward Pass: Compute model prediction $\hat{U}^{t_o} \leftarrow F(U^{t_i})$

 Compute loss $\mathcal{L} \leftarrow g(\hat{U}^{t_o}, U^{t_o})$

Backpropagation: Compute gradients $\nabla_{\theta} \mathcal{L}$ and update θ

end for

return Trained model F

Validation Procedure

Require: Validation dataset $\mathcal{V}$ with n_{val} trajectories with timesteps T , trained model F , validation metric g

Freeze model F

▷ Disable gradient updates

Initialize empty list $B \leftarrow []$

for each validation trajectory in $\mathcal{V}$ **do**

for each (U^{t_i}, U^{t_o}) where $t_i \in [0, T - 1]$ and $t_o \leftarrow t_i + 1$ **do**

 Forward propagate: $\hat{U}^{t_o} \leftarrow F(U^{t_i})$

 Compute loss $\mathcal{L} \leftarrow g(\hat{U}^{t_o}, U^{t_o})$

 Append $\mathcal{L}$ to list B

end for

end for

return $\frac{1}{|B|} \sum_{b \in B} b$

▷ Return the average

Forecasting Procedure

Require: Trained model F , a set of evaluation samples $\mathcal{E}$ with desired targeted time t_o

Initialize empty list $B \leftarrow []$

for each evaluation sample U^{t_i} in $\mathcal{E}$ **do**

 Compute quotient and remainder $q \leftarrow \lfloor (t_o - t_i) / t_h \rfloor$, $t_r \leftarrow (t_o - t_i) \bmod t_h$

for $j \leftarrow 1$ to q **do**

▷ Performing Warp-Unrolling Forecasting (Section 3.3)

 Update $t_c \leftarrow t_i + t_h$

 Forward propagate: $\hat{U}^{t_c} \leftarrow F(U^{t_i})$

 Update $t_i \leftarrow t_c$

end for

if $t_r > 0$ **then**

 Forward propagate: $\hat{U}^{t_o} \leftarrow F(U^{t_i + t_r})$

▷ One last forward pass with the remainder t_r

end if

 Append $\hat{U}^{t_o}$ to list B

end for

return list B

▷ Return predicted samples

C. Data statistics and Hyperparameters for training SCENT on Navier-Stokes Benchmark Datasets

		DATASET NAME		
		NS-3	NS-4	NS-5
DATA STATISTICS	# TRAJECTORIES - TRAIN	256	1000	1000
	# TRAJECTORIES - VALIDATION	64	200	200
	MAXIMUM T	30	30	20
	INITIAL T	10	10	10
	SPATIAL RESOLUTION	(64,64)	(64,64)	(64,64)
	N POINTS - INPUTS (N)	4096	4096	4096
	N POINTS - INPUTS (M)	4096	4096	4096
TRAINING	MAX LR	0.001	0.0006	0.0008
	MIN LR	0	0	0.000008
	LR SCHEDULER	COSINE	COSINE	COSINE
	WARMUP STEPS	0	2000	2000
	BATCH SIZE	100	100	100
	TOTAL STEPS	150000	110000	110000
	OPTIMIZER	ADAMW	ADAMW	ADAMW
	BETA1	0.9	0.9	0.9
	BETA2	0.999	0.999	0.999
	TRAINING TIME HORIZON	5	5	5
	WEIGHT DECAY	0.00001	0.00001	0.00001
MODEL	EMBED DIM	128	128	128
	LATENT DIM	128	128	128
	LINEAR PROJECTION DIM	64	64	64
	# LEARNABLE QUERIES	64	256	256
	# LAYERS - PROCESSOR	2	2	2
	# LAYERS - ENCODER	4	4	4
	# LAYERS - DECODER	4	4	4
	# HEADS	4	4	4
	SPARSE ATTENTION - GROUP SIZE	1	8	8
	FF MULTIPLIER	4	4	4
EMBEDDING	OUTPUT SCALE	0.1	0.1	0.1
	LATENT INIT SCALING (STD)	0.02	0.02	0.02
	FOURIER FEATURES # FREQUENCY BANDS	6	12	12
	FOURIER FEATURES MAX RESOLUTION	20	20	20

D. Data statistics and Hyperparameters for Training SCENT on Simulated Datasets

		DATASET NAME				
		S1	S2	S3	S4	S5
DATA STATISTICS	# TRAJECTORIES - TRAIN	100000	100000	100000	100000	100000
	# TRAJECTORIES - VALIDATION	1000	1000	1000	1000	1000
	MAXIMUM T	50	50	50	50	50
	INITIAL T	1	1	1	1	1
	TEMPORAL SUBSAMPLE STEP SIZE	2	2	2	2	2
	SPATIAL RESOLUTION	(64,64)	(64,64)	(64,64)	(64,64)	(64,64)
	N POINTS - INPUTS (N)	4096	4096	2840	2048	2000
	N POINTS - INPUTS (M)	4096	4096	2840	2048	2000
TRAINING	MAX LR	0.0008	0.0008	0.0008	0.0008	0.0008
	MIN LR	0.00008	0.00008	0.00008	0.00008	0.00008
	LR SCHEDULER	COSINE	COSINE	COSINE	COSINE	COSINE
	WARMUP STEPS	2000	2000	2000	2000	2000
	BATCH SIZE	256	256	256	256	256
	TOTAL STEPS	100000	100000	50000	50000	50000
	OPTIMIZER	ADAMW	ADAMW	ADAMW	ADAMW	ADAMW
	BETA1	0.9	0.9	0.9	0.9	0.9
	BETA2	0.999	0.999	0.999	0.999	0.999
	TRAINING TIME HORIZON	3	3	3	3	3
	WEIGHT DECAY	0.0001	0.0001	0.0001	0.0001	0.0001
MODEL	EMBED DIM	164	164	164	164	164
	LATENT DIM	128	128	128	128	128
	LINEAR PROJECTION DIM	64	64	64	64	64
	# LEARNABLE QUERIES	128	128	128	128	128
	# LAYERS - PROCESSOR	2	2	2	2	2
	# LAYERS - ENCODER	6	6	6	6	6
	# LAYERS - DECODER	6	6	6	6	6
	# HEADS	4	4	4	4	4
	SPARSE ATTENTION - GROUP SIZE	2	2	8	8	8
	FF MULTIPLIER	4	4	4	4	4
EMBEDDING	OUTPUT SCALE	0.1	0.1	0.1	0.1	0.1
	LATENT INIT SCALING (STD)	0.02	0.02	0.02	0.02	0.02
	FOURIER FEATURES # FREQUENCY BANDS	12	12	12	12	12
	FOURIER FEATURES MAX RESOLUTION	20	20	20	20	20

E. Additional Descriptions on Simulated Datasets

This dataset represents an incompressible fluid dynamics system governed by the vorticity transport equation:

$$\frac{\partial \omega}{\partial t} = -\mathbf{u} \cdot \nabla \omega + \nu \Delta \omega + f,$$

where the vorticity is defined as:

$$\omega = \nabla \times \mathbf{u}, \quad \nabla \cdot \mathbf{u} = 0. \quad (2)$$

Here, $\mathbf{u}$ denotes the velocity field, and ω represents the vorticity. Both quantities are defined on a spatial domain with periodic boundary conditions. The parameter ν represents the kinematic viscosity, and f is an external forcing function applied to sustain turbulence.

The input at time t is given as $v_t = \omega_t$. The spatial domain is defined as:

$$\Omega = [-\pi, \pi]^2.$$

The vorticity field is initialized using a Gaussian Random Field (GRF) with parameters: $\alpha = 2.5$, $\tau = 3.0$.

Here, α controls the smoothness of the initial vorticity distribution, while τ determines the correlation length scale of the spatial structures. The external forcing function used in the simulation is:

$$f(x_1, x_2) = -4 \cos(4x_2), \quad (3)$$

where x_2 represents the vertical spatial coordinate.

This forcing function introduces a structured periodic forcing along the vertical direction, promoting rotational flow characteristics. The periodic nature of the cosine function ensures a repeating vortex structure, which sustains turbulence and prevents energy dissipation over time. The negative sign maintains a consistent direction of vorticity input, reinforcing the rotational dynamics within the system. As a result, this setup generates a persistent and well-defined turbulent flow pattern. A Reynolds number of $Re = 100$ is used, indicating a moderately turbulent regime where inertial forces are dominant over viscous forces, allowing for complex vortex interactions while maintaining numerical stability. This is particularly relevant for spatiotemporal learning, as it provides a complex yet structured temporal evolution of the vorticity field, making it an ideal testbed for evaluating models that aim to learn continuous representations of dynamic physical systems. We generate in total 100k trajectories, with 50 time steps per trajectory. Among $T = 50$ data, we use every two steps ($\{0, 2, 4, \dots, 48\}$) for training and validation. The remaining time steps ($\{1, 3, 5, \dots, 49\}$) are reserved for evaluating the model's ability to learn continuous temporal representations.

F. Hyperparameters for Model Scalability Evaluations(Fig. 4(a))

We use dataset S5 for the model scalability evaluations.

		MODEL NAME						
		M1	M2	M3	M4	M5	M6	M7
TRAINING	MAX LR	0.0008	0.0007	0.0006	0.0005	0.0004	0.0003	0.0002
	MIN LR	0.00008	0.00007	0.00006	0.00005	0.00004	0.00003	0.00002
	LR SCHEDULER	COSINE	COSINE	COSINE	COSINE	COSINE	COSINE	COSINE
	WARMUP STEPS	2000	2000	2000	2000	2000	2000	2000
	BATCH SIZE	256	256	256	256	256	256	256
	TOTAL STEPS	50000	50000	50000	50000	50000	50000	50000
	OPTIMIZER	ADAMW	ADAMW	ADAMW	ADAMW	ADAMW	ADAMW	ADAMW
	BETA1	0.9	0.9	0.9	0.9	0.9	0.9	0.9
	BETA2	0.999	0.999	0.999	0.999	0.999	0.999	0.999
	TRAINING TIME	3	3	3	3	3	3	3
	HORIZON	3	3	3	3	3	3	3
	WEIGHT DECAY	0.0001	0.0001	0.0001	0.0001	0.0001	0.0001	0.0001
MODEL	LATENT DIM	128	192	256	384	512	768	1024
	# LEARNABLE QUERIES	128	138	164	192	224	224	256
	# LAYERS - PROCESSOR	2	2	2	2	2	4	6
	# LAYERS - ENCODER	6	6	6	6	6	6	6
	# LAYERS - DECODER	6	6	6	6	6	6	6
	# HEADS - PROCESSOR	4	4	4	6	8	8	12
	# HEADS - ENCODER	2	2	2	2	4	4	4
	# HEADS - DECODER	2	2	2	2	4	4	4
	EMBED DIM	164	164	164	164	164	164	164
	LINEAR- PROJECTION DIM	64	64	64	64	64	64	64
	SPARSE ATTENTION GROUP SIZE	8	8	8	8	8	8	8
	FF MULTIPLIER	4	4	4	4	4	4	4
EMBEDDING	OUTPUT SCALE	0.1	0.1	0.1	0.1	0.1	0.1	0.1
	LATENT INIT SCALING (STD)	0.02	0.02	0.02	0.02	0.02	0.02	0.02
	FOURIER FEATURES	12	12	12	12	12	12	12
	# FREQUENCY BANDS	12	12	12	12	12	12	12
	FOURIER FEATURES MAX RESOLUTION	20	20	20	20	20	20	20

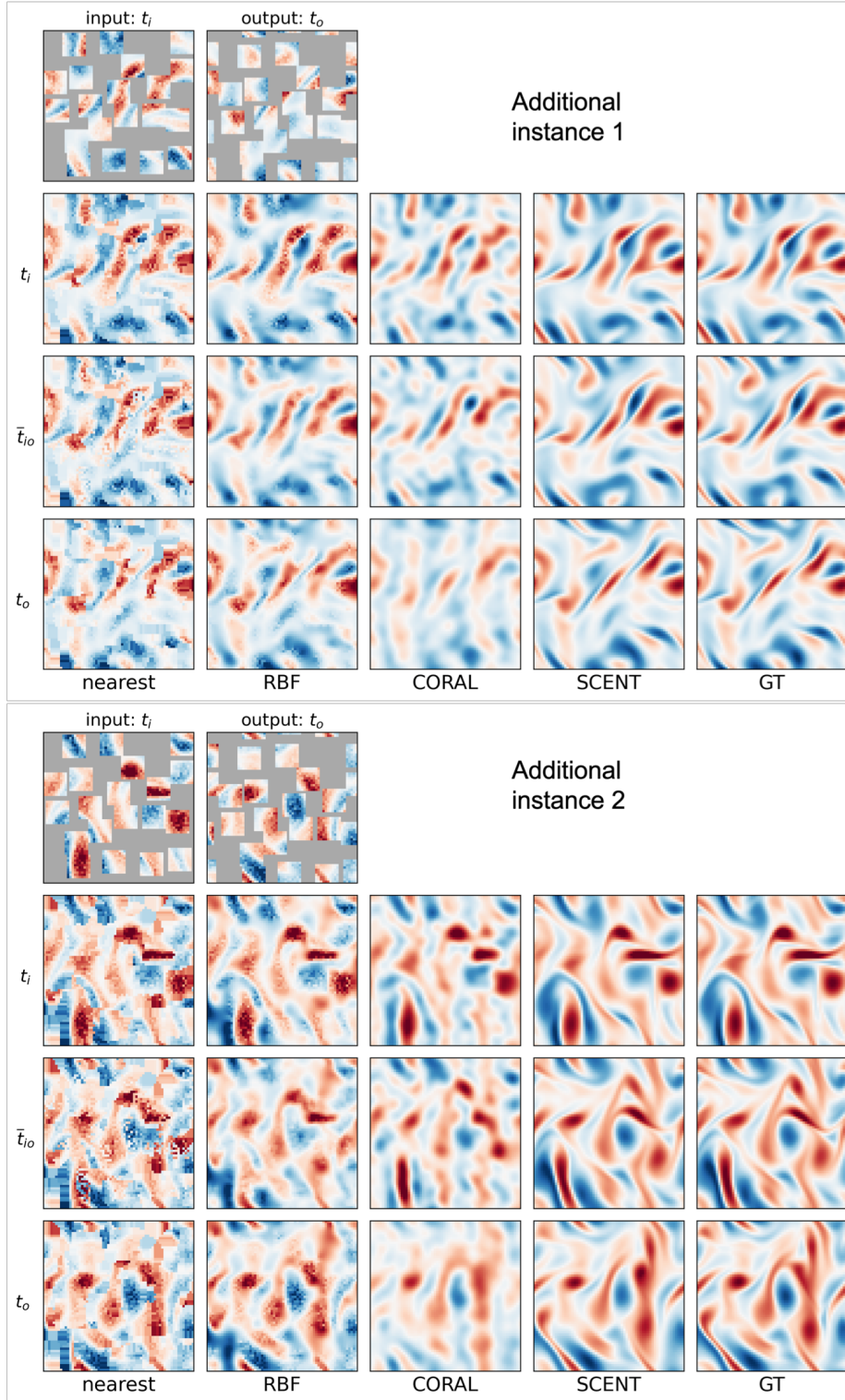
G. Hyperparameters for Dataset Scalability Evaluations (Fig. 4(b))

Other hyperparameters follow Appendix F for each corresponding model size.

MODEL NAME	LATENT DIM	DATASET NAME	# TRAJECTORIES	BATCH SIZE	TOTAL STEPS	MAX LR	MIN LR
M1	128	S5-30	30000	76	50000	0.0008	0.00008
		S5-50	50000	128	50000	0.0008	0.00008
		S5-100	100000	256	50000	0.0008	0.00008
M2	256	S5-30	30000	76	50000	0.0006	0.00006
		S5-50	50000	128	50000	0.0006	0.00006
		S5-100	100000	256	50000	0.0006	0.00006
M3	512	S5-30	30000	76	50000	0.0004	0.00004
		S5-50	50000	128	50000	0.0004	0.00004
		S5 - 100	100000	256	50000	0.0004	0.00004

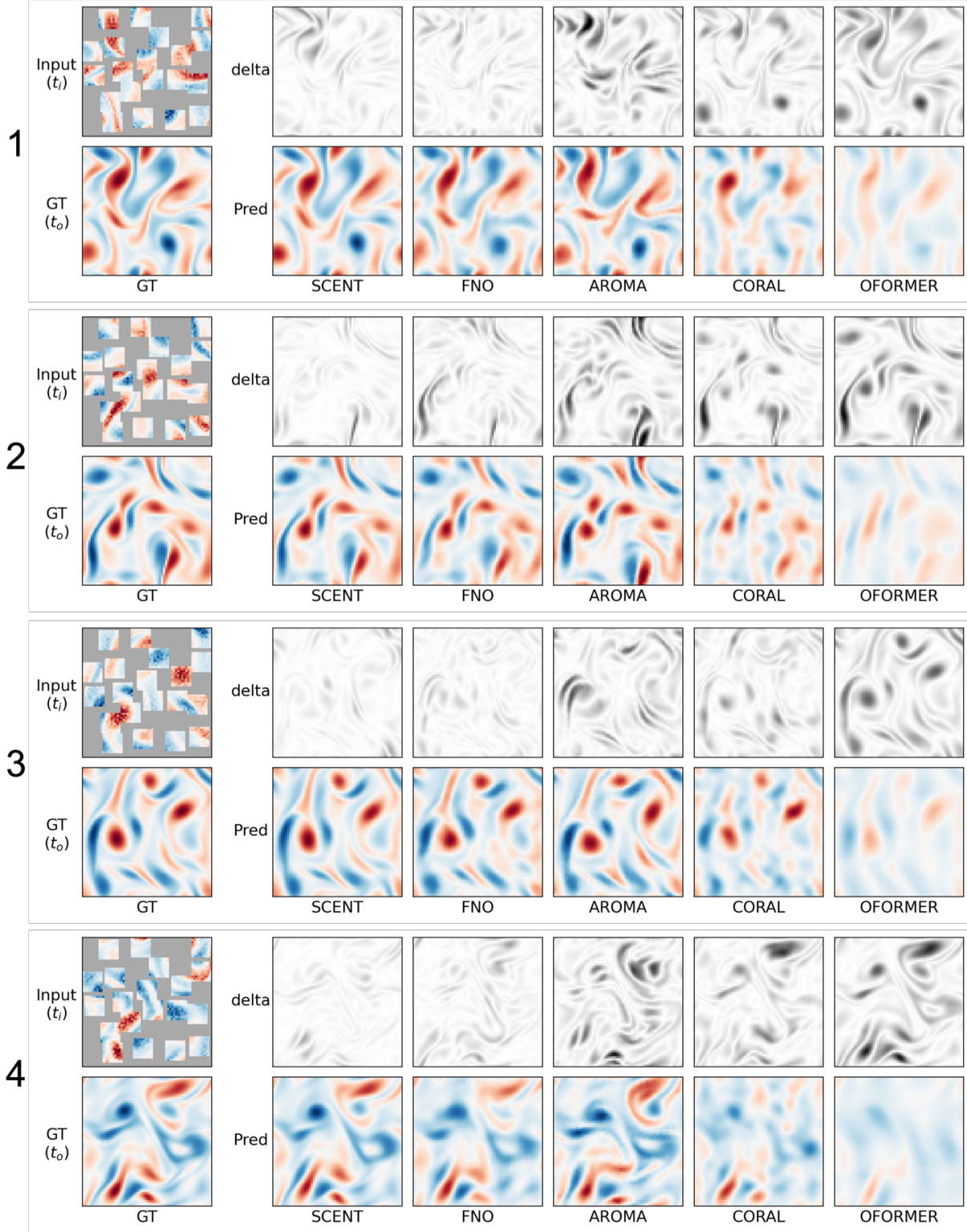
H. Learning Spatiotemporal Continuity: additional results

Two additional instances are illustrated. Please refer to main manuscript Fig. 6 for interpretation.



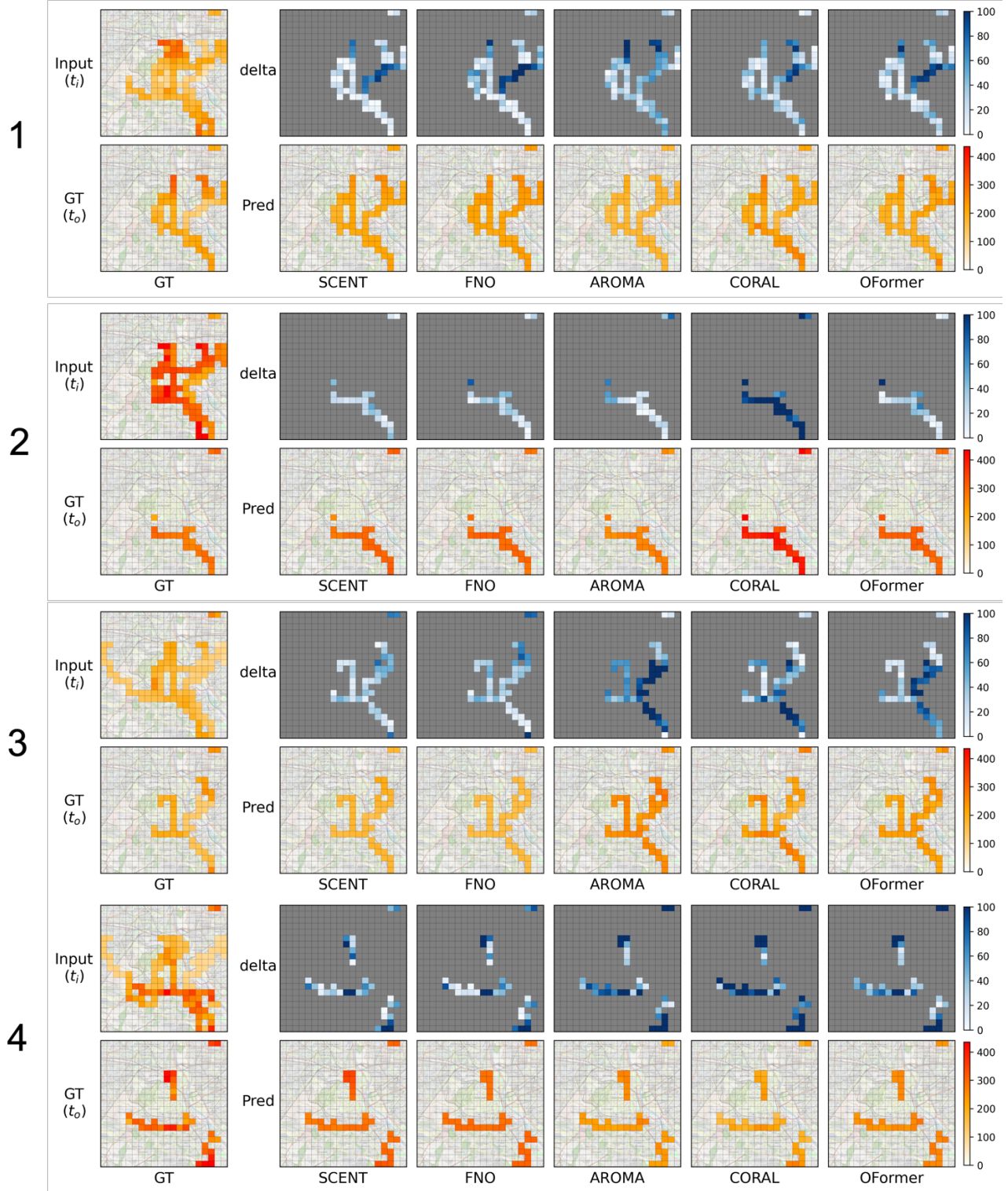
I. Forecasting on S5 dataset: additional results

Four additional instances are illustrated. Please refer to main manuscript Fig. 5(a) for full details and interpretations.



J. Forecasting on AD-B dataset: additional results

Four additional instances are illustrated. Please refer to main manuscript Fig. 5(b) for details and interpretations.



K. AirDelhi AD-B: Comparisons Against Previously Reported Benchmark Performances

Here, we report and compare performances against SCENT and other baselines. Our experiments with conditioned neural fields set new record in AD-B benchmark. Inverse Distance Weighting (IDW) computes the weighted average of all visible samples based on their distances, assigning this value to the held-out locations. Random Forest (RF) is a non-linear model designed to capture complex spatial relationships. It excels in non-linear regression tasks by utilizing an ensemble of decision trees, with the final prediction obtained as the mean output across all trees. XGBoost (XGB) incrementally enhances predictions by combining weak estimators. During training, it employs gradient boosting to optimize performance while sequentially adding new trees. ARIMA (Auto-Regressive Integrated Moving Average) is a statistical model for time-series forecasting that applies linear regression. N-BEATS (Neural Basis Expansion Analysis for Time Series) (Oreshkin et al., 2020) is a deep learning model designed for zero-shot time-series forecasting. Non-Stationary Gaussian Process (NSGP) (Patel et al., 2022) is a recent Gaussian process baseline that models a non-stationary covariance for latitude and longitude, along with a locally periodic covariance for time.

PRIOR REPORTED PERFORMANCES		UPDATED PERFORMANCES	
MODEL	RMSE	MODEL	RMSE
IDW	86.52	FNO	48.79
RF	110.49	OFORMER	70.62
XGBOOST	102.68	CORAL	60.51
NSGP	95.83	AROMA	40.78
ARIMA	148.86	SCENT	<u>44.2</u>
N-BEATS	106.41	-	-

L. Computational Complexity Analysis

Summary. This section provides Big-O time comparisons between SCENT, AROMA, and FNO. In summary, SCENT’s cost is sensitively affected by S which denotes the number of tokens attended within sparse attention layers in CEN (Section 3.2) and CN (Section 3.4). Meanwhile, AROMA with a diffusion transformer backbone is costly if the number of refinement step K and unrolling steps T increase. Lastly, FNO is sensitive to the model width C and also the unrolling steps T . We show that at the scale of the NS-3 experiment (Section 2), SCENT is the most expensive during training among three models, while the gap shrinks for larger unrolling steps. SCENT scales linearly with W , while AROMA scales linearly with K and T . Depending on their values, one could be more expensive than the other. Notably, $W \approx \frac{T}{t_h}$ where t_h is the time horizon of SCENT, thanks to Warp-Unrolling Forecasting (Section 3.3, Fig. 3). Thus, for a longer time forecasting SCENT is more efficient. Please refer to following subsections for big-O time complexity derivations for individual models.

Table 5. Big-O time complexity

MODEL	FORMULA
FNO	$O(T(LN \log NC + LN d^2 C))$
AROMA	$O((2N + 4KT L_2 M + L_1 M) M d)$
SCENT	$O(W L_s N S d + W N M d + W L_m M^2 d)$

L.1. Big-O time complexity of SCENT

For SCENT equipped with:

- **Sparse attention:** L_s layers where each of the N tokens attends to S tokens ($S \ll N$).
- **Cross-attention:** Two layers between N and M tokens.
- **Self-attention:** L_m layers operating on M tokens.

Table 6. Hyperparameters derived from experiments in Section 4.2.

PARAMETER	SYMBOL	VALUE
FOURIER LAYERS IN FNO	L	4
SPATIAL GRID POINTS	N	2000
FOURIER MODES	d	16
MODEL WIDTH	C	60
UNROLLING STEPS (FNO, AROMA)	T	1 OR 20
UNROLLING STEPS (SCENT)	W	1 OR 7
SPARSE ATTENTION LAYERS	L_s	6
SELF-ATTENTION LAYERS	L_m	2
SPARSE ATTENTION TOKENS	S	500
COMPRESSED TOKENS	M	128
LATENT CHANNELS	d	128

Table 7. Sample cost computed using hyperparameters in Table 6

MODEL	UNROLLING $T = 1$		UNROLLING $T = 20$	
	COST	RELATIVE SCALE	COST	RELATIVE SCALE
FNO	1.28E+08	1.0	2.56E+09	1.0
AROMA	2.29E+08	1.78	3.09E+09	1.21
SCENT	8.04E+08	6.28	5.63E+09	2.20

Sparse Attention Blocks

Each token from N attends to only S tokens, reducing the full self-attention cost from $O(N^2d)$ to:

$$O(L_s N S d)$$

Cross-Attention Between N and M Tokens

Each cross-attention operation has cost:

$$O(N M d)$$

With two such layers, the total remains:

$$O(N M d)$$

Self-Attention on M Tokens

A self-attention block on M tokens with L_m layers incurs:

$$O(L_m M^2 d)$$

Final Big-O Complexity

Summing all contributions:

$$O(L_s N S d + N M d + L_m M^2 d)$$

Considering W unrolling during inference, it becomes:

$$O(W L_s N S d + W N M d + W L_m M^2 d)$$

L.2. Big-O for AROMA

AROMA’s computational complexity is provided as following (Serrano et al., 2024):

$$O((2N + 4K L_2 M + L_1 M) M d) \quad (4)$$

where:

- K = Number of refinement steps
- T = Number of autoregressive calls in unrolling
- L_2, L_1 = Number of layers in different parts of the architecture
- N = Number of observations
- M = Number of tokens used to compress information
- d = Number of channels used in the attention mechanism

Diffusion Transformer incurs additional costs due to iterative refinement:

$$O(4KTL_2M^2d) \quad (5)$$

which scales with K (refinement steps) and T (autoregressive calls).

L.3. Big-O for FNO

For an FNO (Li et al., 2020) model with:

- L = Number of Fourier layers
- N = Number of spatial grid points
- d = Number of Fourier modes (spectral channels)
- C = Model width (number of feature channels per layer)
- T = Number of autoregressive unrolling

The key computational operations are:

Fast Fourier Transform (FFT) and Inverse FFT (IFFT)

$$O(LN \log NC) \quad (6)$$

FFT is applied across all feature channels.

Linear Transform in Fourier Space

$$O(LNd^2C) \quad (7)$$

Applies transformations to Fourier modes across all channels.

Total Complexity for Multiple Layers

$$O(T(LN \log NC + LNd^2C)) \quad (8)$$

The unrolling factor T accounts for repeated forward passes in autoregressive prediction.